lock in `pop()`. Because memory allocation is typically quite an expensive operation, this can be very beneficial for the performance of the queue, because it reduces the time the mutex is held, allowing other threads to perform operations on the queue in the meantime.

Just like in the stack example, the use of a mutex to protect the entire data structure limits the concurrency supported by this queue; although multiple threads might be blocked on the queue in various member functions, only one thread can be doing any work at a time. However, part of this restriction comes from the use of `std::queue<>` in the implementation; by using the standard container you now have essentially one data item that's either protected or not. By taking control of the detailed implementation of the data structure, you can provide more fine-grained locking and thus allow a higher level of concurrency.

6.2.3. A thread-safe queue using fine-grained locks and condition variables

In listings 6.2 and 6.3 you have one protected data item (`data_queue`) and thus one mutex. In order to use finer-grained locking, you need to look inside the queue at its constituent parts and associate one mutex with each distinct data item.

The simplest data structure for a queue is a singly linked list, as shown in figure 6.1. The queue contains a *head* pointer, which points to the first item in the list, and each item then points to the next item. Data items are removed from the queue by replacing the head pointer with the pointer to the next item and then returning the data from the old head.

Items are added to the queue at the other end. In order to do this, the queue also contains a *tail* pointer, which refers to the last item in the list. New nodes are added by changing the *next* pointer of the last item to point to the new node and then updating the tail pointer to refer to the new item. When the list is empty, both the head and tail pointers are `NULL`.